```
    "I'm afraid I can't do that"
  end
end

puts Computer.function
```

produces:

```
I'm afraid I can't do that
```

In this example, the object name is self and at the point of the method declaration, self means "The class this method is being declared inside", in this case Computer, so the method is accessible as Computer.function. (You could actually define the method as def Computer.function, and you might see older Ruby code that uses that syntax.) We'll talk more about self in Chapter 22, Metaprogramming, on page 373.

Although class methods are the most common use of this feature, methods can be attached to any object:

```
class Computer
end

mac = Computer.new
pc = Computer.new

def mac.introduction = "I'm a Mac"

def pc.introduction = "I'm a PC"

puts mac.introduction
puts pc.introduction
```

produces:

```
I'm a Mac
I'm a PC
```

In this case, we've attached separate methods to each of the the two instances, so calling introduction on each instance behaves differently.

You'll see this syntax for class methods frequently, and the individual object version of it quite rarely. We'll talk more in Chapter 25, Language Reference: Objects and Classes, on page 465 about why this works, and how class methods behave in Ruby.

Method Parameters

Now that we've defined our new method, we may need to declare some parameters to the method. Parameters are defined using a list of local variable names. Using parentheses around a method's parameters in the definition is optional. The standard convention is to use them when a method has parameters and omit them when it doesn't. Note that if the method is defined using the "endless method" syntax, the parameter list must be surrounded by parentheses, but you don't need to include empty parentheses.

```
def my_new_method(arg1, arg2, arg3)     # 3 parameters
 # Code for the method would go here
end

def my_other_new_method                  # No parameters
 # Code for the method would go here
end
```